

Computer Vision Practical Report:

SHARPENING OF IMAGE USING GRADIENT MASKING

1. AIM

To sharpen an image by computing its gradient magnitude using Sobel operators, smoothing the gradient map, and multiplying it with a Laplacian mask response in OpenCV, display the results, and save the output image.

2. PROCEDURE

- Load the image and convert it to grayscale using `cv2.cvtColor()`.
- Compute the image gradients along the X and Y axes using `cv2.Sobel()` and combine them to form the gradient magnitude map.
- Smooth the gradient mask using a box or Gaussian filter to remove isolated high-frequency noise.
- Multiply the original Laplacian response by the smoothed gradient mask to form a product mask that emphasizes primary edges.
- Add the product mask back to the original image to obtain the final sharpened output, display using Matplotlib, and save using `cv2.imwrite()`.

3. PROGRAM CODE

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Define image path
image_path = 'E25-IMG.jpg'

# Check if file exists in the directory before loading
```

```
if not os.path.exists(image_path):
    print(f"Error: Could not find '{image_path}'. Make sure the image is saved in the same folder as this script!")
else:
    # Load input image
    image = cv2.imread(image_path)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Convert to grayscale for gradient calculation
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 2. Define Sobel Gradient Kernels
    kernel_y = np.array([
        [-1, -2, -1],
        [ 0,  0,  0],
        [ 1,  2,  1]
    ], dtype=np.float32)

    kernel_x = np.array([
        [-1,  0,  1],
        [-2,  0,  2],
        [-1,  0,  1]
    ], dtype=np.float32)

    # 3. Compute horizontal and vertical gradients
    grad_y = cv2.filter2D(gray, cv2.CV_64F, kernel_y)
    grad_x = cv2.filter2D(gray, cv2.CV_64F, kernel_x)

    # 4. Calculate Gradient Magnitude (Edge Mask)
    gradient_magnitude = cv2.magnitude(grad_x, grad_y)

    # 5. Add gradient mask back to original image to sharpen
    gradient_mask = np.uint8(np.clip(gradient_magnitude, 0, 255))
    gradient_mask_3ch = cv2.cvtColor(gradient_mask, cv2.COLOR_GRAY2RGB)
```

```
# Sharpened Output = Original + Gradient Mask
sharpened_rgb = cv2.add(image_rgb, gradient_mask_3ch)

# 6. Display input and output side-by-side on a SINGLE PAGE
plt.figure(figsize=(10, 5))

# Input Image
plt.subplot(1, 2, 1)
plt.imshow(image_rgb) # Fixed variable name from E25-IMG_rgb
plt.title('Original Image')
plt.axis('off')

# Sharpened Output
plt.subplot(1, 2, 2)
plt.imshow(sharpened_rgb)
plt.title('Gradient Sharpened Output')
plt.axis('off')

plt.tight_layout()
plt.show()
```

4. OUTPUT

Original Image



Gradient Sharpened Output



5. RESULT

The Python script executed successfully. The input image was loaded.